Application of Spatial Data Classification Based on K-means and Voronoï Diagram

Moubaric KABORE
ED-ST, LAMDI
University Nazi BONI
BOBO DIOULASSO
BURKINA FASO
kmoubaric24@gmail.com

Abdoulaye SERE
ED-ST, LAMDI
University Nazi BONI
BOBO DIOULASSO
BURKINA FASO
abdoulaye.sere@recifaso.org

ZOUNGRANA Béné-wendé Odilon Isaïe
ED-ST, LAMI
University Joseph KI-Zerbo
Ouagadougou
BURKINA FASO
zoung.odilon@gmail.com

***Abstract*—Efficient management of spatial data, such as geographic coordinates, shapes, and map objects, requires the use of spatial indexes. These data structures optimize query, search, and spatial analysis operations on large volumes of georeferenced information, interconnected by spatial relationships. This paper focuses on the design and implementation of a spatial index adapted to a regular or irregular spatial grid to accelerate the processing of spatial data. The objective is to improve the performance of spatial queries such as intersections, inclusions, and neighborhoods while minimizing computational and memory costs. Spatial indexing thus plays a key role in geographic information systems (GIS), spatial databases, and real-time geolocation applications.**

***Index Terms*—Spatial index, Geographic Information Systems (GIS), regular grid, irregular grid, Big Data.**

I. Introduction

In a context where spatial data plays an increasing role in sectors as diverse as urban planning, logistics, precision agriculture, and even the distribution of medical resources, the ability to process it efficiently has become a major challenge. This data, generally georeferenced, is not limited to a simple position on the Earth's surface: it is often interconnected by complex spatial relationships such as proximity, inclusion, or intersection.

For example, the location of pharmaceutical points of sale, drug stocks, or health coverage areas constitutes a type of spatial information whose analysis is essential for decision-making. Such cases illustrate the diversity of fields in which spatial data is used and highlight the need for effective indexing mechanisms.

Managing this large-scale data requires appropriate structures to optimize the execution of spatial queries, reducing processing costs and improving performance. Spatial indexing structures such as square grids and Voronoi diagrams are therefore essential in the context of geographic information systems (GIS) or spatial databases. This work is a continuation of the work of Sere and al. in [**2**] and Kabore and al. in [**1**], which respectively use indexing by superposition of a regular grid (8-neighborhood) and an irregular V(K(x)) grid that combines kmeans and Voronoi. Mehdi and al. [**4**] also used VoR-tree, an indexing system that combines the Voronoi diagram for neighborhoods and the R-tree for traversal.

This work aims to design and analyze an efficient spatial indexing method, applicable to a wide range of domains, without being limited to a specific case. The objective is to show how these techniques allow for faster processing of complex data, taking into account the spatial relationships between objects.

II. Preliminary

A. Statement of the problem

With the exponential growth of spatial data from sensors, GPS systems, map databases, and various business applications, systems must manage ever-increasing volumes of georeferenced information. This data, in addition to being localized, presents complex spatial relationships (proximity, inclusion, intersection), essential for its processing and analysis.

Conventional relational database management systems, however, are unsuitable for efficiently handling this spatial data. Standard indexes, such as B-trees, do not take into account the geometric structure or topological relationships between objects. As a result, spatial queries become costly in terms of time and resources, especially when they must be executed on large volumes or in real time.

Furthermore, in several application areas (urban planning, logistics, healthcare, the environment, etc.), it is becoming crucial to perform efficient spatial searches to enable rapid and relevant decision-making. For example, identifying the nearest drug distribution points or analyzing the geographic coverage of a service requires optimized queries.

It is therefore necessary to design and implement efficient spatial indexing structures capable of quickly processing complex and heterogeneous spatial data, while remaining applicable to a variety of domains.

B. Spatial and georeferenced data

Spatial data represents objects or phenomena located on the Earth's surface. They are said to be georeferenced

when they are associated with a geographic coordinate system (such as latitude/longitude) or projected (e.g., UTM). This data allows entities to be located in physical space and is fundamental to geographic information systems (GIS).

There are three main types of geometries:

- Points: precise location (e.g., an ATM)
- Lines: linear objects (e.g., roads, rivers)
- Polygons: delimited areas (e.g., neighborhoods, agricultural plots)

These objects can contain attributes (name, type, date, etc.) and are stored in spatial databases.

1) Spatial relations: Spatial objects do not exist in isolation. They maintain spatial relationships with each other, which reflect their organization or interaction in space. These relationships are essential for advanced analyses.

The main spatial relationships are:

- Intersection: two objects share an area (e.g., a river crosses a municipality)
- Inclusion/Containment: one object is inside another (e.g., a pharmacy in a health zone)
- Proximity: distance between objects (e.g., two nearby hospitals)
- Adjacency/Contiguity: objects that share a boundary (e.g., two neighboring neighborhoods)

These relationships can be modeled using topological, metric, or directional approaches.

2) Spatial queries: These spatial queries allow a database to be queried by taking into account the position and geometry of objects. Unlike traditional queries, they incorporate spatial criteria. Some researchers have already investigated this, such as Chen in [**5**], who uses the Range query and Nearest neighbor queries to test the effectiveness of the spatial index. Examples of spatial queries:

- Range query: "Find all hospitals within a 5 km radius"
- Nearest neighbor: "What is the nearest health center?"
- Spatial join: "Associate each pharmacy with the health zone in which it is located"

Spatial queries can be object queries, attribute queries, or hybrid queries [**6**]. These queries are essential in the fields of health, urban planning, transportation, and logistics.

3) Spatial data structures: Spatial data structures are organizational mechanisms that allow geometric objects to be indexed to optimize access and search [**1**, **2**, **4**, **5**]. The main structures are:

- R-tree: hierarchizes objects within bounding rectangles, suitable for objects of various shapes (widely used in spatial databases)
- Quadtree: recursively divides space into quadrants, effective for gridded or regularly distributed data
- Grid: divides space into fixed cells, useful for area-based searches
- Irregular grid.

These structures greatly improve the performance of spatial queries, especially when working with millions of objects. Sere and al. in [**2**], Chen and al. in [**5**] used regular grids for spatial indexing work to minimize query time. Kabore and al. in [**1**] uses an irregular grid, namely the Voronoi diagram, for spatial indexing.

4) Geographic Information Systems (GIS) and Spatial Databases: GIS are tools that allow you to store, analyze, visualize, and query spatial data. Examples include QGIS, ArcGIS, and Google Earth Engine.

Spatial databases (such as PostGIS, Oracle Spatial, and MongoDB with a geospatial extension) allow you to directly store and index spatial objects and their relationships. They are essential for projects handling large amounts of geographic data.

III. Description of the method

Choosing the indexing structure is a crucial step in spatial data processing. It directly affects the performance of spatial queries, system scalability, and result accuracy. The goal is to select a structure that is suited to the nature of the data, spatial density, and the type of queries to be optimized. The method focuses on regular grids and Voronoi diagrams. The R-tree is also a method that can be used for spatial indexing.

A. Indexing of regular grids

The superposition of a regular grid on the space allows to link each data to its belonging grid. The relationship between the cells maintains a relationship between the data of each cell as illustrated in figures 2 and 3. This relationship allows to perform spatial indexes in order to minimize the time of complex queries. Consider any bounded domain noted Q to which a mesh has been applied. The mesh of the domain Q is said to be structured if and only if from a single one of its meshes it is possible to reconstruct the whole identical mesh. It is characterized by a cell that repeats itself identically and ordered in [**3**, **7**]. Depending on whether we are in 1D or 2D dimension, Kabore and al. in [**1**] described regular grids as follows:

- 1D mesh : the mesh is carried out on the right of the abscissa whose domain is a segment [AB] in figure 1. This segment is partitioned into m cells with a constant pitch C which is equal to $\frac{B-A}{m}$. Then $X_1 = A, X_{i+1} = X_i + C$ and $X_m = B$ with $i = 1, 2 \ldots m$.
  The relationship $X_{i+1} = X_i + C$ expresses the connectivity link between two cells.

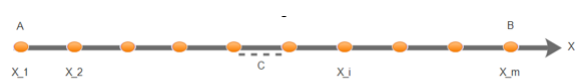

Fig. 1: 1D mesh

2D mesh : for a rectangular domain (a, b) x (c, d) we have:

- $X_1 = a, X_i + 1 = X_i + C$ with C the pitch of the mesh in the direction X; $C = \frac{b-a}{m}$ and $i = 1, 2...m$
- $Y_1 = a, Y_j + 1 = Y_J + K$ with K the pitch of the mesh in the Y direction; $K = \frac{b-a}{m}$ and $j = 1, 2...m$

This mesh is composed of pairs of transverse lines which intersect at the nodes of the mesh and a cell which is repeated with the same number of nodes around a vertex. Each node is obtained by the coordinates (i, j) formed by the indices of the transverse lines. The decrementation and/or the incrementation of the coordinates (i, j) of a node makes it possible to locate its neighbors. Then the incrementation and the decrementation constitute the relations of connectivities between the different cells.

A	1	2	3	4	5	6
B	7	8	9	10	11	12
C	13	14	15	16	17	18
D	19	20	21	22	23	24

Fig. 2: Square grid

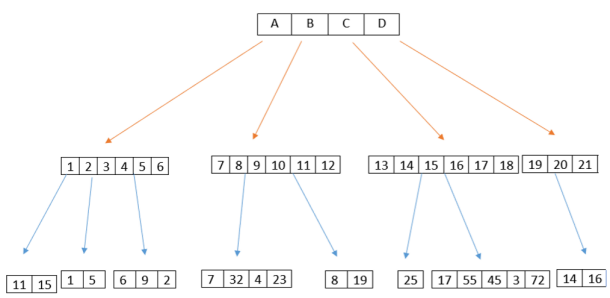

Fig. 3: R-Tree

B. Indexing irregular grids

In Sere and al. [**2**], we used regular grids, whose superposition with homogeneous data is more suitable. In Kabore and al. [**1**], we used irregular grids, namely the Voronoi diagram, whose application to heterogeneous data is more suitable. The application of V(K(x)) described by Kabore and al. in [**1**] generates an overlay of the Voronoi diagram on heterogeneous spatial data. Each Voronoi cell indexes data in its area. The different cells maintain a link. These links are also maintained between the data in each cell. The application of V(K(x)) is therefore a spatial indexing. Figure 4 shows the representation of the application of V(K(x)) to obtain irregular cells. They are represented by an R-tree in figure 5.

IV. Experimental result

The characteristics of the computer used are:

Operating System: Windows 11 Professionnel 64-bit (10.0, Build 26100) (26100.ge_release.240331-1435)

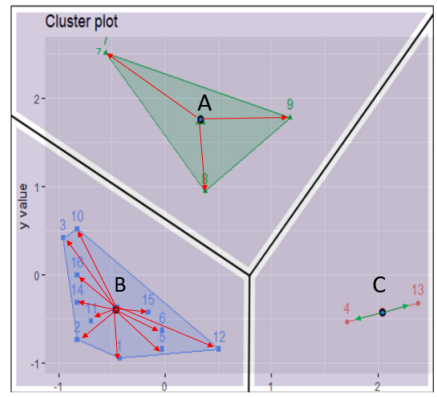

Fig. 4: Mesh generated by Voronoi and k-means

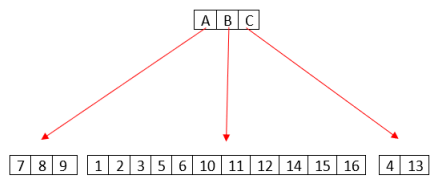

Fig. 5: Representation of the mesh by an R tree

System Manufacturer: HP
System Model: HP ENVY Laptop 17-cg1xxx
BIOS: F.35 (type: UEFI)
A mechanical hard drive (HDD) of 1 TB
A 256 GB NVMe SSD
Processor: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz (8 CPUs), 2.8GHz
Memory: 16384MB RAM
Available OS Memory: 16146MB RAM
Page File: 11005MB used, 17428MB available
DirectX Version: DirectX 12
DirectX Database Version: 1.7.3
Mux Target GPU: dGPU

Vehicle sales position data is stored in a MongoDB database. Here's how the data is represented for both the regular and irregular grid overlays.

{ "_id": {
"$oid": "68287164d0b94c6fa49b4d88" },
"name_business": "Used car seller",
"type_brand": "particular car",
"quantity": 1,
"city": "Ouagadougou",
"borough": "borough 5",
"sector": "29.0",
"location_site": {
"coordinates": [-1.4923, 12.3683]
}
"cluster": 7 }

an update that adds the neighbors field to each document
{ "_id": {
"$oid": "68287164d0b94c6fa49b4d88" },
"name_business": "Used car seller",
"type_brand": "particular car",
"quantity": 1,
"city": "Ouagadougou",
"borough": "borough 5",
"sector": 29.0,
"location_site": {
"coordinates": [-1.4923, 12.3683]
"type": "Point" }
"cluster": 7,
"neighbors": ["2", "3", "4", "6", "8", "10", "11", "12"] }

{ "_id": {
"$oid": "67d9e322fb2479980b03f25e" },
"name_business": "Used car seller",
"type_brand": "particular car",
"quantite": 1,
"city": "Ouagadougou",
"borough": "borough 5",
"sector": "29.0",
"locate_site": {
"coordinates": [
-1.492267545,
12.36833806]
} ,
"cluster": 8. }
An update that adds the neighbors field to each document
{ "_id": "oid_0166",
"name_business": "Used car seller",
"type_brand": "particular car",
"quantite": 1,
"city": "Ouagadougou",
"borough": "borough 5",
"sector": 23,
"locate": {
"type": "Point",
"coordinates": [
-1.493,
12.3519
]
}
, "cluster": 1,
"neighbors": ["5", "7", "9"] }
The neighbor field of the regular record contains the list of all neighbors in cluster 7. The neighbors field of the irregular record contains the list of all neighbors in cluster 1.

A. The performance of the index based on regular grids and 8-neighborhoods

The data has undergone several treatments. In particular, grouping by cluster, in which each document contains a cluster field that indicates its cluster membership, and neighborhood relationships, in which each document contains a neighbors field that indicates the 8-neighborhood of its cluster. Thus, several indexes can be created, namely:

- the index on identifiers: an index on the _id field is automatically created; it is not a spatial index,
- the index on coordinates: a 2dsphere index has been applied to the location_site field for geospatial queries with the command "db.Auto2.createIndex({ "locate_site": "2dsphere" }) " named location_site_2dsphere,
- the cluster index: a cluster index is created using the command "db.Auto2.createIndex({ "cluster": 1 })" named cluster_1,
- the neighbor index: a neighborhood index is created using the command "db.Auto2.createIndex({ "neighbors": 1 })" named neighbors_1,
- the cluster and sector index: a combined index on clusters and sectors was created using the command "db.Auto.createIndex({ "cluster": 1, "sector": 1 })" named cluster_1_sector_1.

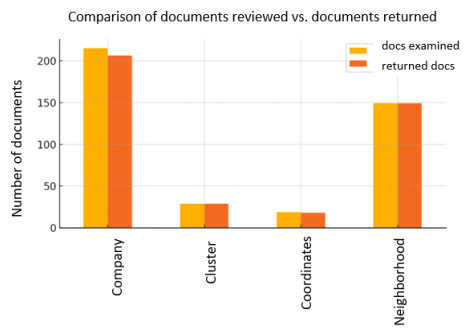

Fig. 6: Comparison of indexes (number of documents examined and returned)

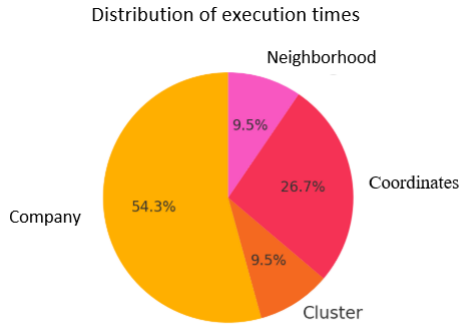

Fig. 7: Comparing indexes (execution time)

Queries targeting each index were formulated in Python to compare their performance in the 1 algorithm. These queries are:

- query 1, which targets the name_company index, searches for all documents where the name_company field exactly matches the specified value.

Method	Time (s)	Documents Scanned	Documents Returned	Plan
Query 1: company	0.0057	215	206	COLLSCAN
Query 2: cluster	0.0010	29	29	FETCH
Query 3: coordinates	0.0028	19	18	GEO_NEAR_2DSPHERE
Query 4: neighborhood	0.0010	149	149	FETCH

TABLE I: Comparison of index performance

- query 2, which targets the cluster index_1, searches for all documents where the cluster field is equal to 3.
- query 3, which targets the location_site_2dsphere index, searches for documents within 2 km of the point [-1.4995, 12.3422].
- query 4, which targets the neighbors_1 index, searches for all documents where the cluster field is one of the values [2, 4, 6, 7, 8].

The metrics used are: query execution time in ms, the number of documents examined and the number of documents returned as presented in table I and figures 6 and 7. An indexing is said to be efficient if it can examine only the documents concerned by its search with a less expensive execution time. That is to say that the number of documents examined corresponds to or is slightly higher than the number of documents returned. Looking at table I, the histogram 6 comparing the documents examined with the returned documents, and the pie chart 7 of execution times, it appears that:

- the company index uses the COLLSCAN read plan; it examines the entire database and takes an enormous amount of time, reducing its performance compared to other indexes;
- the cluster index uses the FETCH read plan; it reads only 29 documents relevant to the query with minimal read time, confirming high performance;
- the coordinate index uses the GEO_NEAR_2DSPHERE read plan; it reads 19 documents and returns 18 documents relevant to the query with an average read time; it has average performance;
- the neighborhood index uses the FETCH read plan; it reads only 149 documents affected by the query with a very minimal reading time, confirming very high performance.

The cluster and neighborhood indexes examine and return 29 and 149 documents, respectively, in 9.5% of the time each. These are the most efficient. The neighborhood index examines and returns more documents than the cluster index in the same time; it is the most efficient of all indexes.

B. Performance evaluation of the Voronoi cell-based and neighborhood-based index

The data underwent several treatments. In particular, grouping by cluster, in which each document contains a cluster field indicating its cluster membership, and neighborhood relationships, in which each document contains a neighbors field indicating the neighbors by vertex and by stop of its cluster. Thus, several indexes can be created, namely:

- the index on identifiers: an index on the _id field is automatically created; it is not a spatial index,
- the index on coordinates: a 2dsphere index has been applied to the location field for geospatial queries with the command "db.Auto.createIndex({ "location": "2dsphere" })" named location_2dsphere,
- the index on neighbors: a neighborhood index is created using the command "db.Auto.createIndex({ "neighbors": 1 })" named neighbors_1,
- the neighborhood and cluster index: a combined index on clusters and neighbors has been created using the command "db.Auto.createIndex({ "cluster": 1, "neighbors": 1 })" named cluster_1_neighbors_1.

Queries targeting each index were formulated in Python in the 2 algorithm in order to compare their performance. These queries are:

- query 1, which targets the location_2dsphere index, uses only a geospatial filter to find the documents closest to a given point within a 50 km radius,

Algorithm 1: Comparative analysis of MongoDB query performance

Data: Database `BigData`, collection `Auto2`, reference point $(-1.4995,\ 12.3422)$
Result: Statistics on the performance of various queries

```
Connect to MongoDB database
Initialize collection Auto2
Define the queries to analyze:
    Query 1 ← company = "Used car seller"
                                    // Exact name filter
    Query 2 ← cluster = 3
                                    // Filter by cluster ID
    Query 3 ← location within 2 km of the
      reference point
                                    // Spatial query
    Query 4 ← cluster ∈ {2, 4, 6, 7, 8}
                                    // Neighborhood-based filter
foreach (name, query) in the list of queries do
    Start ← current time
    Plan ← explain() on the query
    End ← current time
    Stats ← executionStats from Plan
    Print query name
    Print "Execution time (s):" + (End - Start)
    Print "Documents examined:" +
      Stats.totalDocsExamined
    Print "Documents returned:" +
      Stats.nReturned
    Print "Execution plan used:" +
      Stats.executionStages.stage
```

TABLE II: Comparison of query processing performance by index

	Execution Time (ms)	Documents Returned	Documents Scanned	Keys Scanned
Algo 1: Query 1 (location_2dsphere)	4.47	215	215	318
Algo 2: Query 2 (neighbors_1 + cluster_1_neighbors_1)	0.01	167	167	251
Algo 3: Query 3 (location_2dsphere + neighbors_1)	4.48	167	430	318
Algo 4: Query 4 (neighbors_1)	0.00	167	167	251
Difference (Query 3 - Query 1)	0.01	-48	215	0

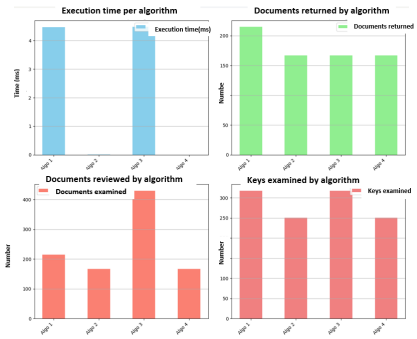

Fig. 8: Index Performance Comparison Histogram

- query 2, which targets the combined neighbors_1 and cluster_1_neighbors_1 index, filters documents belonging to clusters 7 or 8 and documents that are neighbors of at least clusters [7, 8, 9],
- query 3, which targets the combined location_2dsphere and neighbors_1 index, uses a geospatial filter to find the documents closest to a given point within a 50 km radius and another filter to find documents that are neighbors of at least clusters [7, 8, 9],
- query 4, which targets the neighbors_1 index, filters documents that are neighbors of at least clusters [7, 8, 9].

The metrics used are: execution time in ms, number of returned documents, number of examined documents and number of examined keys as presented by table II and figure 8. Each query is executed 100 times to obtain reliable metrics (via the run_query function) as described by the algorithm 2 . An index is said to be efficient if it can examine only the documents concerned by its search. That is to say that the number of examined documents corresponds to the number of returned documents. With regard to table II and histogram 8, the combined index neighbors_1, cluster_1_neighbors_1 and the neighbors_1 index each examine 167 documents and return 167 documents in a record time of 0.01 ms and 0.00 ms respectively. These are high-performance indexes. The combined location_2dsphere and neighbors_1 index examines up to 430 documents, which is apparently twice the database, and returns only 167 documents. This is the least efficient index. The location_2dsphere index examines 215 documents, which is the entire database, and returns all 215 documents, probably because the query covers all documents. However, its execution time is 4.47 ms, which is slow for 215 documents. This is an average-performance index.

Algorithm 2: Execution and Comparison of MongoDB Queries with Different Indexes

```
Data: BigData database, Auto collection, 4 predefined MongoDB queries
Result: Average execution metrics and query comparison
Connection ← MongoClient("mongodb://localhost:27017/");
db ← Connection["BigData"];
collection ← db["Auto"];
// Define a function to run a query multiple times and collect statistics
Function RunQuery(query, num_runs) :
    TimeList ← empty list;
    ReturnList ← empty list;
    DocsList ← empty list;
    KeysList ← empty list;
    for i ← 1 to num_runs do
        ExplainOutput ← collection.find(query).explain();
        Stats ← ExplainOutput["executionStats"];
        TimeList.add(Stats["executionTimeMillis"]) ;          // Execution time in ms
        ReturnList.add(Stats["nReturned"]) ;                  // Returned documents
        DocsList.add(Stats["totalDocsExamined"]) ;            // Scanned documents
        KeysList.add(Stats["totalKeysExamined"]) ;            // Scanned keys
    return AvgTime: mean(TimeList),
    AvgReturn: mean(ReturnList),
    AvgDocs: mean(DocsList),
    AvgKeys: mean(KeysList);
// Define the queries
query_1 ← {location near [-1.529, 12.295] within 50 km};
query_2 ← {cluster = 7 or 8, or neighbors in [7, 8, 9]};
query_3 ← {combination of queries 1 and 2};
query_4 ← {neighbors in [7, 8, 9]};
// Execute the queries
result_1 ← RunQuery(query_1, 100);
result_2 ← RunQuery(query_2, 100);
result_3 ← RunQuery(query_3, 100);
result_4 ← RunQuery(query_4, 100);
// Display the results
Print("Query 1 results: ", result_1);
Print("Query 2 results: ", result_2);
Print("Query 3 results: ", result_3);
Print("Query 4 results: ", result_4);
// Quick comparison between Query 3 and Query 1
timeDiff ← result_3.AvgTime − result_1.AvgTime;
docsDiff ← result_3.AvgDocs − result_1.AvgDocs;
Print("Time difference (Query 3 - Query 1): ", timeDiff, " ms");
Print("Difference in scanned documents (Query 3 - Query 1): ", docsDiff);
```

Conclusion and perspectives

Faced with the exponential growth of spatial data and the increasing complexity of their relationships, the development of efficient indexing methods is an essential lever to ensure efficient, fast and resource-efficient processing. This work is part of this dynamic by exploring indexing approaches adapted to both regular and irregular grids, while drawing on recent advances in the field, such as those of Sere, Kabore and Mehdi. By promoting structures such as square grids, Voronoi diagrams and VoR-trees, it highlights their ability to optimize complex spatial queries — intersection, inclusion, neighborhood — within geographic information systems or distributed databases. The proposed approach thus offers a generic solution, applicable to various usage contexts, ranging from urban infrastructure management to the intelligent distribution of health services, ultimately contributing to more responsive and relevant spatial decision-making.

References

[1] Moubaric KABORE, Béné-wendé Odilon Isaïe Zoungrana, and Abdoulaye SERE. "Classification of Spatial Data Based on K-means and Voronoï Diagram." International Journal of Advanced Computer Science and Applications, Volume 15, No 7 (2024). http://dx.doi.org/10.14569/IJACSA.2024.01507139

[2] Abdoulaye SERE, Jean Serge Dimitri OUATTARA, Didier BASSOLE, Jose Arthur OUEDRAOGO and Moubaric KABORE, “A Framework for Data Research in GIS Database using Meshing Techniques and the Map-Reduce Algorithm” International Journal of Advanced Computer Science and Applications(IJACSA), Vol. 12, No. 3, 2021.

[3] S. J. Owen, A survey of unstructured mesh generation technology, Proceedings, Seventh International Meshing Roundtable, Sandia National Laboratories, Albuquerque, NM, 1998, pp. 239–267.

[4] Mehdi Sharifzadeh et Cyrus Shahabi. Vor-tree: R-trees with voronoi diagrams for efficient processing of spatial nearest neighbor queries. Proceedings of the VLDB Endowment, 2010, vol. 3, no 1-2, p. 1231-1242.

[5] Chen, Guangsheng, Pei Nie, and Weipeng Jing. "A novel query method for spatial data in mobile cloud computing environment." Wireless Communications and Mobile Computing (2018): https://doi.org/10.1155/2018/1059231

[6] X. Meng, P. Li and X. Zhang, "A Personalized and Approximated Spatial Keyword Query Approach". In IEEE Access, Volume 8, pages 44889-44902, (2020). https://doi.org/10.1109/ACCESS.2020.2977996

[7] Duchaine, François. Génération de maillage structuré par krigeage avec raffinement elliptique local. Diss. École de technologie supérieure, 2004.

[8] Deshayes, Michel, and Jean-Pierre Chery. "SIG, définitions et contraintes de mise en place." Forêt méditerranéenne, vol. 21, no 1, p. 67-69 (2000).